

Package ‘spaDesign2’

June 17, 2026

Title spaDesign2: Pilot-Based Power Analysis and Sample-Size Determination for Multi-Sample Spatial Transcriptomics

Version 1.0.1

Author Jungmin Shin [aut, cre],
Juan Xie [aut],
Dongjun Chung [aut]

Maintainer Jungmin Shin <c16267@gmail.com>

Description Provides a pilot-based framework for statistical power analysis and sample-size determination in comparative, multi-sample spatial transcriptomics studies. A cohort-level generative model is learned from pilot data through three layers: a baseline-anchored Binomial-logit model for domain composition, a Fisher-Gaussian kernel mixture model for spatial geometry (Mukhopadhyay, Li and Dunson (2020) <[doi:10.1111/rssb.12390](https://doi.org/10.1111/rssb.12390)>), and a Gaussian-process expression model fitted with nearest-neighbor approximations (Saha and Datta (2018) <[doi:10.1002/sta4.184](https://doi.org/10.1002/sta4.184)>). Synthetic cohorts are generated under user-specified effect sizes, spatial domains are recovered with a pilot-guided extension of 'BANKSY' (Singhal et al. (2024) <[doi:10.1038/s41588-024-01664-3](https://doi.org/10.1038/s41588-024-01664-3)>), and endpoint-specific power is estimated under a full generate-recover-test pipeline for a spatially adjusted differential-expression test and a baseline-anchored compositional test, both using a minimum-effect 'TREAT' margin (McCarthy and Smyth (2009) <[doi:10.1093/bioinformatics/btp053](https://doi.org/10.1093/bioinformatics/btp053)>). Monte-Carlo power estimates are summarized with monotone shape-constrained additive models to recommend per-group sample sizes. The framework supports both replicated and single-pilot designs and has been evaluated across the 'Visium', 'Visium HD', and 'Stereo-seq' platforms.

Depends R (>= 4.1.0)

License GPL (>= 3)

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 8.0.0

LinkingTo Rcpp,
RcppArmadillo

Imports BRISC, clue, Directional, dplyr, FNN, ggplot2, MASS, Matrix,
matrixStats, mclust, MCMCpack, methods, patchwork, Rcpp, scam, stats, limma

Suggests data.table, fields, ggrepel, knitr, mgcv, RANN, readr, rmarkdown, RSpectra, Seurat, SingleCellExperiment, SpatialExperiment, spatialLIBD, stringr, testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

biocViews Software, SingleCell, Spatial, Transcriptomics, GeneExpression, DifferentialExpression, Clustering, Bayesian

Collate 'LOR.R' 'PowerEvaluation.R' 'RcppExports.R' 'SaLFC.R' 'createSpaDesign2Object.R' 'data.R' 'estimation.R' 'featureSelection.R' 'fn_FGEM_aux.R' 'fn_aux_base.R' 'lambda_tuning.R' 'pBANSKY.R' 'rearrangement.R' 'simulateSpaDesign2.R' 'spaDesign2-package.R'

Contents

addTestingResult	3
as_powerLOR	3
as_powerSaLFC	4
createSpaDesign2Object	4
estimateCompositionParams_v2	5
estimateExpressionParams_v2	6
estimateGeometryParams_v2	7
estimate_lfc_threshold_from_pilot	8
estimate_lor_tau_from_pilot	10
evaluatePowerLOR	11
evaluatePowerSaLFC	13
featureSelection	15
featureSelectionStable	16
LOR	17
makeCustomGeneSets	19
mini_custom_genes	21
mini_obj_features	21
mini_obj_fitted	22
mini_pilot_data_list	23
mini_spaDesign2_obj	23
pBANSKY	24
plot_spaDesign2_missing-method	25
plot_spaDesign2Tuning	27
plotPowerCurveLOR	27
plotPowerCurveSaLFC	28
print.powerLOR	28
print.powerSaLFC	29
rearrangeSyntheticToPilot	29
SaLFC	30
simulateSpaDesign2	33
spaDesign2-class	35
summary.powerLOR	35
summary.powerSaLFC	36
summary.spaDesign2Tuning	36
syntheticData	37
testingResult	37
tuneSpaDesignLambdas	38

addTestingResult	<i>Add a Single Endpoint Result</i>
------------------	-------------------------------------

Description

Convenience helper that inserts or overwrites one named endpoint in the `testing_result` slot, leaving existing endpoints intact.

Usage

```
addTestingResult(object, name, result)
```

Arguments

<code>object</code>	A <code>spaDesign2</code> object.
<code>name</code>	Character. Endpoint key (e.g., "salfc", "composition").
<code>result</code>	The result object to store under <code>name</code> .

Value

The updated `spaDesign2` object.

as_powerLOR	<i>Tag a LOR power grid as class "powerLOR"</i>
-------------	---

Description

Tag a LOR power grid as class "powerLOR"

Usage

```
as_powerLOR(x, alpha = 0.05)
```

Arguments

<code>x</code>	Data frame from <code>evaluatePowerLOR</code> .
<code>alpha</code>	Numeric. The level used, stored for display.

Value

`x` with class `c("powerLOR", ...)` and an `alpha` attribute.

as_powerSaLFC	<i>Tag a SaLFC power grid as class "powerSaLFC"</i>
---------------	---

Description

Tag a SaLFC power grid as class "powerSaLFC"

Usage

```
as_powerSaLFC(x, alpha = 0.05)
```

Arguments

x	Data frame from evaluatePowerSaLFC.
alpha	Numeric. The (FDR) level used, stored for display.

Value

x with class c("powerSaLFC", ...) and an alpha attribute.

createSpaDesign2Object	<i>Create spaDesign2 Object</i>
------------------------	---------------------------------

Description

Creates a spaDesign2 object from a pre-processed list of pilot data. If the dataset contains only single replicates (K=1) globally or per group, this function can automatically generate pseudo-replicates via minor random perturbations to ensure downstream multi-sample estimators function correctly.

Usage

```
createSpaDesign2Object(
  pilot_list,
  generate_pseudo_reps = TRUE,
  noise_level = 0.05
)
```

Arguments

pilot_list	A list of processed samples. Each element must contain counts (genes x spots), coords (with columns 'x', 'y', 'domain'), and group.
generate_pseudo_reps	Logical. If TRUE, expands any group with K=1 to K=3 by adding minor spatial and count perturbations. Default is TRUE.
noise_level	Numeric. Fractional uniform noise added to counts for pseudo-replicates. Default is 0.05 (+/- 5%).

Value

A spaDesign2 object ready for parameter estimation.

Examples

```
## Not run:
# CRAN-safe way to load package data from a nested directory
# (It is recommended to place such files in 'inst/extdata/' for R packages)
data_path <- system.file("extdata", "Visium", "human_brain",
                        "visium_human_brain_pilot_data_list.RData",
                        package = "spaDesign2")

load(data_path)

# Create the spaDesign2 object
spaDesign2_obj <- createSpaDesign2Object(pilot_data_list)

## End(Not run)
```

estimateCompositionParams_v2

Estimate Composition Parameters (Multi-Sample)

Description

Estimates parameters for the composition model via Binomial GLM and empirical pilot moments.
Requires K>1 replicates per group.

Usage

```
estimateCompositionParams_v2(
  object,
  target_domain,
  reference_domain,
  prob_floor = 1e-06,
  verbose = TRUE
)
```

Arguments

object	A spaDesign2 object containing pilot_data.
target_domain	Character. Target domain name.
reference_domain	Character. Reference domain name.
prob_floor	Numeric. Probability clamp for numerical safety.
verbose	Logical. Print progress.

Value

Updated spaDesign2 object with params_composition.

Examples

```
## Not run:
data("mini_obj_features", package = "spaDesign2")
spaDesign2_obj <- estimateCompositionParams_v2(
  object = mini_obj_features,
  target_domain = "WM",
  reference_domain = "Layer6"
)

## End(Not run)
```

```
estimateExpressionParams_v2
```

Estimate Expression Parameters (Multi-Sample Optimized)

Description

Estimates gene expression parameters across multiple samples. Uses Rcpp/OpenMP to pre-compute biological variance, grand means, and residuals, massively speeding up the spatial estimation loop.

Usage

```
estimateExpressionParams_v2(
  object,
  target_domain,
  reference_domain,
  genes_to_use = NULL,
  n_neighbors = 15,
  n_cores = 1,
  verbose = TRUE
)
```

Arguments

<code>object</code>	A spaDesign2 object.
<code>target_domain</code>	Character. Target domain name.
<code>reference_domain</code>	Character. Reference domain name.
<code>genes_to_use</code>	Character vector or NULL. Genes to evaluate.
<code>n_neighbors</code>	Integer. Number of neighbors for spatial modeling (default: 15).
<code>n_cores</code>	Integer. Number of cores for parallel processing.
<code>verbose</code>	Logical. Print progress messages.

Value

Updated spaDesign2 object with `params_expression` populated.

Examples

```
## Not run:
data("mini_obj_features", package = "spaDesign2")
spaDesign2_obj <- estimateExpressionParams_v2(
  object = mini_obj_features,
  target_domain = "WM",
  reference_domain = "Layer6",
  genes_to_use = sets$G_svg,
  n_cores = 8
)

## End(Not run)
```

estimateGeometryParams_v2

Estimate Spatial Geometry Parameters (Multi-Sample)

Description

Estimates parameters for domain geometry via FGKMM pooling across multiple samples.

Usage

```
estimateGeometryParams_v2(
  object,
  M_candidates = 3:6,
  iter_max = 500,
  tol = 0.01,
  n_cores = 1,
  verbose = TRUE,
  min_spots = 30,
  trim_outliers = TRUE,
  trim_k = 10L,
  trim_quantile = 0.98,
  gc_each_fit = FALSE,
  store_criteria = TRUE,
  worker_sources = NULL,
  max_pool_spots = 5000L
)
```

Arguments

object	A spaDesign2 object.
M_candidates	Integer vector. Candidate M values for FGKMM.
iter_max	Integer. Max EM iterations.
tol	Numeric. EM convergence tolerance.
n_cores	Integer. Number of PSOCK cores.
verbose	Logical. Print progress.
min_spots	Integer. Minimum spots per domain per sample.

trim_outliers Logical. Density-based outlier trimming before FGKMM fitting.
 trim_k Integer. KNN for density estimation.
 trim_quantile Numeric in (0,1).
 gc_each_fit Logical. Trigger garbage collection each fit.
 store_criteria Logical. Store BIC criteria table.
 worker_sources Character vector. Optional R scripts to source on PSOCK workers.
 max_pool_spots Integer. Max pooled centered spots for M selection.

Value

Updated spaDesign2 object with params_geometry.

Examples

```
## Not run:
data("mini_obj_features", package = "spaDesign2")
spaDesign2_obj <- estimateGeometryParams_v2(
  object = mini_obj_features,
  n_cores = 3
)

## End(Not run)
```

estimate_lfc_threshold_from_pilot

Estimate Gene-Specific SaLFC TREAT Thresholds from Pilot Data

Description

Fast plug-in estimator (no bootstrap) of gene-specific LFC thresholds τ_g for the TREAT-style SaLFC test, using only pilot data. SaLFC is run once on the pilot samples with `lfc_threshold = 0`; τ_g is then defined from the pilot effect size and its uncertainty in a direction-aware way:

$$\tau_g = \max(\text{effect_ref}_g, z_q \cdot \text{se}_g),$$

where `effect_ref` depends on alternative and $z_q = \Phi^{-1}(q)$.

This module is normally invoked automatically by `SaLFC(..., lfc_threshold = "pilot")`, but it can also be called directly and its result passed as `lfc_threshold`.

Usage

```
estimate_lfc_threshold_from_pilot(
  object,
  genes,
  target_domain = "WM",
  reference_domain = "Layer6",
  rho_TR = 0,
  adjust_method = "bonferroni",
  test_method = "t",
  alternative = c("greater", "less", "two.sided"),
```

```

q = 0.75,
min_tau = 0,
cap_effect = NULL,
max_spots_per_domain = 5000L,
verbose = TRUE,
return_table = FALSE,
test_fn = NULL,
B = 200,
shrink_to_global = 0.25,
seed = 1
)

```

Arguments

object	A spaDesign2 object with @pilot_data.
genes	Character vector of (null) genes, e.g. the stable/null set.
target_domain	Character. Target domain label.
reference_domain	Character. Reference domain label.
rho_TR	Numeric. Assumed Target-Reference biological correlation (< 1).
adjust_method	Multiple-testing method passed through to SaLFC.
test_method	SaLFC group comparison: "t" or "wilcoxon".
alternative	Test direction: "greater" (default), "less", "two.sided".
q	Numeric in (0,1). Standard-normal quantile level for z_q .
min_tau	Numeric. Lower bound for τ_g .
cap_effect	Numeric scalar, named numeric vector, or NULL. Optional upper cap on τ_g (per gene if named).
max_spots_per_domain	Integer. Per-sample spot cap before running SaLFC on the pilot (bounds memory). The global RNG state is restored on exit.
verbose	Logical.
return_table	Logical. If TRUE return a data.frame, else a named vector.
test_fn	Function. The SaLFC test to run on the pilot. If NULL, resolves SaLFC by name. When invoked via <code>SaLFC(..., lfc_threshold = "pilot")</code> , the running test function is passed automatically, so renaming it does not break this module.
B, shrink_to_global, seed	Deprecated and ignored; retained for backward compatibility.

Value

A named numeric vector τ_g carrying an `attr(, "global_tau")`, or a data.frame when `return_table = TRUE`.

See Also

[SaLFC](#)

estimate_lor_tau_from_pilot

Estimate the Composition TREAT Threshold (τ) from Pilot Data

Description

Fast plug-in estimator of the scalar TREAT threshold τ for the sample-level composition test [LOR](#), mirroring the SaLFC pilot estimator but on the log-odds-ratio scale. LOR is run once on the pilot data with $\tau = 0$ (centered null); τ is then defined direction-aware from the pilot effect and its uncertainty:

$$\tau = \max(\text{effect_ref}, z_q \cdot \text{se}),$$

where effect_ref depends on alternative, the effect is the de-baselined $\tilde{\delta} = \hat{\delta} - \delta_0$, and $z_q = \Phi^{-1}(q)$. Returns one scalar per (target, reference) pair.

Normally invoked automatically by `LOR(..., tau = "pilot")`; can also be called directly. The pilot run uses the ground-truth `coords$domain` (pilot data has no `hat_d`).

Usage

```
estimate_lor_tau_from_pilot(
  object,
  target_domain = "WM",
  reference_domain = "Layer6",
  test_method = c("t", "wilcoxon"),
  alternative = c("greater", "less", "two.sided"),
  q = 0.75,
  min_tau = 0,
  cap_effect = NULL,
  eps = 0.5,
  verbose = TRUE,
  test_fn = NULL
)
```

Arguments

<code>object</code>	A <code>spaDesign2</code> object with <code>@pilot_data</code> and <code>@params_composition</code> .
<code>target_domain, reference_domain</code>	Character domain labels.
<code>test_method</code>	"t" (default) or "wilcoxon". Only "t" yields an se for the plug-in; "wilcoxon" falls back to <code>effect_ref</code> alone (se term = 0).
<code>alternative</code>	"greater" (default), "less", or "two.sided".
<code>q</code>	Numeric in (0,1). Standard-normal quantile level for z_q .
<code>min_tau</code>	Numeric. Lower bound for τ .
<code>cap_effect</code>	Numeric or NULL. Optional upper cap on τ .
<code>eps</code>	Numeric > 0. Continuity correction forwarded to LOR.
<code>verbose</code>	Logical.
<code>test_fn</code>	Function. The composition test to run on the pilot. If NULL, resolves LOR by name. When invoked via <code>LOR(..., tau = "pilot")</code> , the running test function is passed automatically.

Value

A numeric scalar τ ($\geq \text{min_tau}$).

See Also

[LOR](#)

evaluatePowerLOR

Evaluate Composition (LOR) Power over a Grid

Description

Grid simulation over sample sizes (K) and composition effect sizes to evaluate the power and empirical size of the sample-level log-odds-ratio test [LOR](#). Per replicate the pipeline is `simulateSpaDesign2() -> pBANSKY() -> LOR()`. The effect size is the case-vs-control shift in the `target_domain` abundance, injected through the simulator's composition scenario (`delta_pp`, in percentage points, or `target_prop_case`); expression DE is switched off (`DE_lfc = 0`, `de_genes = character(0)`) so the swept signal is purely compositional.

Unlike the per-gene SaLFC test, LOR returns a SINGLE test per (target, reference) pair, so there is no multiple testing within a replicate: **power = rejection rate** across replicates at each (K , effect), and the effect = 0 row is the **empirical size**. LOR counts spots by `coords$hat_d` when present (recovered clusters) and otherwise by `coords$domain`; `eval_on` selects which. The composition TREAT threshold τ does not depend on the effect size, so under `null_type = "treat"` the pilot τ is estimated ONCE (a scalar) and reused. A progress bar is shown and simulation errors are skipped.

Usage

```
evaluatePowerLOR(
  object,
  K_grid,
  delta_grid,
  n_sim = 30,
  seed = 1,
  genes,
  target_domain,
  reference_domain,
  effect_type = c("target_prop_case", "delta_pp"),
  eval_on = c("hat_d", "domain"),
  null_type = c("treat", "two_sample"),
  tau = "auto",
  tau_q = 0.75,
  test_method = c("t", "wilcoxon"),
  alternative = c("greater", "two.sided", "less"),
  alpha = 0.05,
  eps = 0.5,
  rearrange = FALSE,
  jitter_frac = 0.3,
  k_nn = 50L,
  n_cores = 1L,
  scenario_base = list(kappa_m = 1, kappa_bio = 1, delta_rho = 1, lambda_cond = 0.5),
```

```

clustering_args = list(lambda = 0.5, k_neighbors = 20, scale_pilot_coords = FALSE,
  scale_sim_coords = FALSE, hard_guidance = FALSE, n_pcs = 20, do_hungarian = TRUE,
  use_G = TRUE, balance_mode = "energy_proportional"),
  tuning_obj = NULL,
  ...
)

```

Arguments

object	A spaDesign2 object (fitted: pilot, params, @params_composition).
K_grid	Numeric vector of samples per group (e.g. seq(3, 8)).
delta_grid	Numeric vector of composition effect sizes. When effect_type = "delta_pp" these are percentage-point shifts of the target proportion (0 = null); when "target_prop_case" they are the absolute case-group target proportions in (0, 1).
n_sim	Integer. Replications per grid point.
seed	Integer. Master seed.
genes	Character vector of genes to simulate (e.g. sets\$G_svg); needed only so pBANSKY has counts to cluster. Expression DE is off.
target_domain, reference_domain	Domain labels.
effect_type	"delta_pp" or "target_prop_case" (default).
eval_on	Label column LOR counts by: "hat_d" (recovered clusters, default) or "domain" (ground-truth oracle).
null_type	"treat" (default; centered at the pilot baseline $\delta_0 = \beta_1$ with margin τ) or "two_sample" ($H_0 : \bar{L}_1 = \bar{L}_0$, τ forced to 0).
tau	"auto" (default; pilot plug-in scalar via estimate_lor_tau_from_pilot) or a numeric scalar. Used only when null_type = "treat".
tau_q	Numeric in (0,1). Quantile for the auto τ .
test_method	"t" (Welch, default) or "wilcoxon".
alternative	"greater" (default), "two.sided", or "less".
alpha	Numeric. Rejection threshold.
eps	Numeric > 0. Continuity correction forwarded to LOR.
rearrange	Logical. Morph synthetic coords onto pilot geometry first. Default FALSE: LOR counts by label and is invariant to the spatial rearrangement, so this only costs time.
jitter_frac, k_nn	Rearrangement tuning (used only when rearrange).
n_cores	Integer. Cores for simulate/cluster.
scenario_base	List. Base scenario; any delta_pp, target_prop_case, or DE_lfc here are dropped (they are controlled by the sweep).
clustering_args	List passed to pBANSKY (must contain lambda). NB: for composition power the recovered hat_d must preserve per-sample target abundance, so avoid size-equalizing balance modes.

tuning_obj Optional spaDesign2Tuning object from [tuneSpaDesignLambdas](#). When supplied, lambda_cond (scenario) and lambda (clustering_args) are taken *per effect size* from optimal_params\$LOR\$by_effect (matched on target_prop_case; nearest value with a warning if no exact match). Only applied when effect_type = "target_prop_case"; otherwise the scenario_base/clustering_args values are used. Default NULL.

... Forwarded simulate tuning: graph_k, gp_rank, gp_jitter, cond_k_nn.

Value

A data.frame of per-replicate rejection rows, tagged with class "powerLOR" (see [print.powerLOR](#), [summary.powerLOR](#), [plotPowerCurveLOR](#)).

See Also

[evaluatePowerSaLFC](#), [LOR](#), [tuneSpaDesignLambdas](#)

evaluatePowerSaLFC	<i>Evaluate SaLFC Power over a Grid</i>
--------------------	---

Description

Grid simulation over sample sizes (K) and effect sizes (DE_lfc) to evaluate the power and FDP of the SaLFC endpoint. Per replicate the pipeline is simulateSpaDesign2() -> pBANSKY() -> rearrangeSyntheticToPilot() -> SaLFC(). Synthetic coordinates are morphed onto the pilot domain geometry by eval_domain_col ("hat_d" by default), and SaLFC selects target/reference by the SAME column, so the spatial variance τ^2 is computed on a consistent basis. Pilot TREAT thresholds (lfc_threshold = "auto") are estimated once per DE_lfc and shared across replicates. A progress bar is shown and simulation errors are skipped gracefully.

Usage

```
evaluatePowerSaLFC(
  object,
  K_grid,
  lfc_grid,
  n_sim = 30,
  seed = 1,
  G_svg,
  G_null,
  G_spike,
  target_domain,
  reference_domain,
  eval_domain_col = c("hat_d", "domain"),
  rearrange = TRUE,
  jitter_frac = 0.3,
  k_nn = 100L,
  test_method = c("t", "wilcoxon"),
  adjust_method = c("bonferroni", "BH"),
  alternative = c("greater", "less", "two.sided"),
  alpha_fdr = 0.05,
```

```

lfc_threshold = "auto",
lfc_q = 0.75,
spatial_mode = c("original", "basis"),
n_cores = 1L,
scenario_base = list(target_prop_case = NULL, delta_pp = NULL, kappa_m = 1, kappa_bio =
  1, delta_rho = 1, lambda_cond = 0.5),
clustering_args = list(lambda = 0.5, k_neighbors = 20, scale_pilot_coords = FALSE,
  scale_sim_coords = FALSE, hard_guidance = FALSE, n_pcs = 20, do_hungarian = TRUE,
  use_G = TRUE, balance_mode = "energy_proportional"),
tuning_obj = NULL,
...
)

```

Arguments

object	A spaDesign2 object (fitted: pilot + params).
K_grid	Numeric vector of samples per group (e.g. seq(3, 8)).
lfc_grid	Numeric vector of effect sizes (DE_lfc, on the count-model log scale used by simulateSpaDesign2).
n_sim	Integer. Replications per grid point.
seed	Integer. Master seed.
G_svg	Character. All genes to simulate (sets\$G_svg).
G_null	Character. Null genes to evaluate (sets\$G_null).
G_spike	Character. True spike-in DE genes (sets\$G_spike).
target_domain, reference_domain	Domain labels.
eval_domain_col	Label column for morphing AND testing: "hat_d" (recovered clusters, default) or "domain" (ground truth).
rearrange	Logical. Morph synthetic blobs onto pilot geometry first.
jitter_frac, k_nn	Rearrangement tuning (used when rearrange).
test_method	"t" or "wilcoxon".
adjust_method	"BH" or "bonferroni" (default).
alternative	SaLFC direction: "greater" (default), "less", "two.sided".
alpha_fdr	Numeric. Rejection threshold for power/FDP.
lfc_threshold	"auto" (pilot plug-in), a scalar, or a named vector.
lfc_q	Numeric. Quantile for the auto threshold.
spatial_mode	"original" or "basis".
n_cores	Integer. Cores for simulate/cluster (keep modest under nesting).
scenario_base	List. Base scenario (composition kept neutral by default so the swept effect is purely expression DE).
clustering_args	List. Passed to pBANSKY (must contain lambda).

tuning_obj Optional spaDesign2Tuning object from [tuneSpaDesignLambdas](#). When supplied, the conditional-texture weight lambda_cond (in the simulation scenario) and the clustering weight lambda (in clustering_args) are taken *per effect size* from the matching row of optimal_params\$SaLFC\$by_effect (matched on DE_lfc; nearest value with a warning if there is no exact match), overriding the values in scenario_base/clustering_args. Default NULL.

... Forwarded simulate tuning: graph_k, gp_rank, gp_jitter, cond_k_nn.

Value

A data.frame of per-replicate power/FDP rows, tagged with class "powerSaLFC" (see [print.powerSaLFC](#), [summary.powerSaLFC](#), [plotPowerCurveSaLFC](#)).

See Also

[evaluatePowerLOR](#), [SaLFC](#), [tuneSpaDesignLambdas](#)

featureSelection

Select Domain-Informative Genes via Multi-Context Meta-Analysis

Description

Identifies robust marker genes by evaluating spatial expression across multiple biological contexts (e.g., NAT vs. CRC). This meta-analysis mitigates representation bias, ensuring the final feature space reflects the full transcriptomic heterogeneity.

Usage

```
featureSelection(
  object,
  rep_sample_ids = NULL,
  logfc_cutoff = 0.5,
  mean_in_cutoff = 1,
  max_num_gene = 20,
  n_cores = 1,
  combine_method = c("union", "intersection"),
  verbose = TRUE
)
```

Arguments

object A spaDesign2 object.

rep_sample_ids Character vector of representative samples. At least one sample per condition should be provided. Defaults to the first sample if NULL.

logfc_cutoff Numeric. Minimum absolute log fold change (default: 0.5).

mean_in_cutoff Numeric. Minimum mean log-expression (default: 1.0).

max_num_gene Integer. Max top genes to extract per domain (default: 20).

n_cores Integer. CPU cores for parallel processing (default: 1).

combine_method Character. "union" (default) or "intersection".

verbose Logical. Print progress.

Value

Updated spaDesign2 object with integrated features.

Examples

```
## Not run:
data("mini_spaDesign2_obj", package = "spaDesign2")
spaDesign2_obj <- featureSelection(
  object = mini_spaDesign2_obj,
  logfc_cutoff = 0.5,
  mean_in_cutoff = 0.8,
  max_num_gene = 3,
  n_cores = 8
)

## End(Not run)
```

featureSelectionStable

Select Spatially and Group-wise Stable Genes

Description

Highly optimized version utilizing pre-filtering and vectorized operations to bypass the computational bottleneck of gene summaries and nested loops.

Usage

```
featureSelectionStable(
  object,
  control_group_value = "0",
  check_group_diff = TRUE,
  case_group_value = "1",
  max_group_logfc_cut = 0.1,
  mean_global_cut = 0.3,
  pct_in_cut = 0.05,
  min_domain_prop = 0.8,
  max_abs_logfc_cut = 0.2,
  cv_cut = 0.3,
  max_num_gene = 50,
  use_low = TRUE,
  n_cores = 1,
  verbose = TRUE
)
```

Arguments

object A spaDesign2 object.

control_group_value Character. Value indicating the control group (default "0").

check_group_diff Logical. Check expression difference between Control and Case (default TRUE).

case_group_value	Character. Value indicating the case group (default "1").
max_group_logfc_cut	Numeric. Maximum allowed absolute logFC between Control and Case (default 0.1).
mean_global_cut	Numeric. Minimum global mean log-expression (default 0.3).
pct_in_cut	Numeric. Minimum detection rate within a domain (default 0.05).
min_domain_prop	Numeric. Minimum proportion of domains that must meet pct_in_cut (default 0.8).
max_abs_logfc_cut	Numeric. Maximum allowed spatial logFC variance (default 0.2).
cv_cut	Numeric. Maximum allowed Robust CV across domain means (default 0.3).
max_num_gene	Integer. Maximum number of genes to return (default 50).
use_low	Logical. Use robust lower-bound logFC instead of standard logFC (default TRUE).
n_cores	Integer. Number of cores for parallel processing.
verbose	Logical. Print progress.

Value

Updated spaDesign2 object with stable_genes and stable_gene_stats populated.

Examples

```
## Not run:
data("mini_spaDesign2_obj", package = "spaDesign2")
spaDesign2_obj <- featureSelectionStable(
  object = mini_spaDesign2_obj,
  max_group_logfc_cut = 0.1,
  mean_global_cut = 0.1,
  min_domain_prop = 0.2,
  max_abs_logfc_cut = 0.1,
  cv_cut = 0.2,
  max_num_gene = 10,
  use_low = FALSE,
  n_cores = 8
)

## End(Not run)
```

Description

Computes per-sample $L_k = \log((y_k + \epsilon)/(b_k + \epsilon))$ and compares Case vs Control at the sample level using a Welch t-test or a Wilcoxon rank-sum test, for a single (target, reference) domain pair. Here y_k and b_k are the spot counts of the target and reference domains in sample k .

Two null modes, controlled by `null_type`:

"`treat`" (**default**) TREAT-style test (Smyth, 2009) centered on the pilot baseline $\delta_0 = \beta_1$.

- $\tau = 0$ (centered null): $H_0 : \hat{\delta} = \delta_0$.
- $\tau > 0$ (TREAT): $H_0 : |\hat{\delta} - \delta_0| \leq \tau$.

"`two_sample`" Naive two-sample test; the pilot baseline is ignored and $\delta_0 = 0$, i.e. $H_0 : \bar{\delta}_1 = \bar{\delta}_0$.

The threshold τ may be a numeric scalar or the string "`pilot`", in which case it is estimated from `object@pilot_data` via `estimate_lor_tau_from_pilot()` for this single (target, reference) pair.

Usage

```
LOR(
  processed_data,
  object,
  target_domain = "WM",
  reference_domain = "Layer6",
  test_method = c("t", "wilcoxon"),
  alternative = c("greater", "two.sided", "less"),
  null_type = c("treat", "two_sample"),
  tau = 0,
  pilot_tau_args = list(),
  eps = 0.5,
  verbose = TRUE
)
```

Arguments

<code>processed_data</code>	List of samples. Each element is a list with <code>coords</code> (containing <code>hat_d</code> or <code>domain</code>) and <code>meta</code> (with <code>group</code> in (0/1) and optionally <code>sample_id</code>).
<code>object</code>	A <code>spaDesign2</code> object with <code>@params_composition</code> (and <code>@pilot_data</code> when <code>tau = "pilot"</code>).
<code>target_domain</code>	Character. Target domain label.
<code>reference_domain</code>	Character. Reference domain label.
<code>test_method</code>	"t" (Welch, default) or "wilcoxon".
<code>alternative</code>	"greater" (default), "two.sided", or "less".
<code>null_type</code>	"treat" (default) or "two_sample".
<code>tau</code>	Numeric scalar ≥ 0 , or the string " <code>pilot</code> ". TREAT threshold (used only when <code>null_type = "treat"</code>); ignored (forced to 0) for " <code>two_sample</code> ". If " <code>pilot</code> ", the threshold is estimated on the fly.
<code>pilot_tau_args</code>	Named list of extra arguments forwarded to <code>estimate_lor_tau_from_pilot()</code> when <code>tau = "pilot"</code> (e.g. <code>list(q = 0.75, min_tau = 0.05)</code>). <code>object</code> , <code>target_domain</code> , <code>reference_domain</code> , <code>test_method</code> , <code>alternative</code> , and <code>eps</code> are supplied automatically.
<code>eps</code>	Numeric > 0 . Continuity correction (default 0.5).
<code>verbose</code>	Logical (default TRUE).

Value

A named list with elements `null_type`, `target`, `reference`, `K0`, `K1`, `delta_hat`, `delta0`, `delta_tilde`, `tau`, `eps`, `Z_bar0`, `Z_bar1`, `y`, `m`, `sample_id`, `group`, `stat`, `df`, `p_value`, `method` (and `stat_treat` in TREAT mode).

References

Smyth, G.K. (2009). Testing significance relative to a fold-change threshold is a TREAT. *Bioinformatics*, 25(6), 765-771.

See Also

[evaluatePowerLOR](#), [estimate_lor_tau_from_pilot](#)

Examples

```
## Not run:
data("mini_obj_fitted", package = "spaDesign2")
data("mini_custom_genes", package = "spaDesign2")

## One-point LOR power calculation (single design point: K = 5,
## case target proportion = 0.7). Expression DE is switched off.
sim <- simulateSpaDesign2(mini_obj_fitted, n_sample_per_group = 5,
  scenario_settings = list(DE_lfc = 0, target_prop_case = 0.7),
  target_domain = "WM", genes_to_simulate = mini_custom_genes$G_svg,
  de_genes = character(0), seed_base = 1)

syn <- pBANSKY(sim, mini_obj_fitted@pilot_data, lambda = 0.5, do_hungarian = TRUE)
syn <- setNames(lapply(syn, function(s) list(coords = s$coords,
  meta = data.frame(sample_id = s$sample_id, group = s$group))),
  vapply(syn, function(s) s$sample_id, character(1)))

res <- LOR(syn, mini_obj_fitted, target_domain = "WM",
  reference_domain = "Layer6", null_type = "treat", tau = "pilot")

## single-test rejection at this design point
res$p_value <= 0.05

## End(Not run)
```

Description

Constructs customized gene sets (SVG, Null, and Spike) based on a specified target and reference domain within a selected pilot sample. Identifies the top differentially expressed genes from a pool of stable candidate genes.

Usage

```
makeCustomGeneSets(
  object,
  G_svg_base,
  G_stable,
  target_domain,
  reference_domain,
  sample_index = 1L,
  n_de = 5L,
  verbose = TRUE
)
```

Arguments

object	A spaDesign2 object.
G_svg_base	Character vector. Base marker genes known a priori.
G_stable	Character vector. Stable genes identified via feature selection.
target_domain	Character. The name of the target domain (e.g., "WM").
reference_domain	Character. The name of the reference domain (e.g., "Layer6").
sample_index	Integer or Character. The index or name of the pilot sample to use (default: 1L).
n_de	Integer. Number of top differentially expressed genes to extract (default: 5L).
verbose	Logical. Print progress and summary messages (default: TRUE).

Value

A list containing:

G_svg	Character vector. Union of G_svg_base and G_stable.
G_null	Character vector. Unique stable genes.
G_spike	Character vector. Top n_de differentially expressed genes.
TR_stats	Data frame. Mean expressions and differences for all tested genes.

Examples

```
## Not run:
data("mini_spaDesign2_obj", package = "spaDesign2")
G_svg_base <- mini_spaDesign2_obj@params_expression$top_genes
G_stable <- mini_spaDesign2_obj@params_expression$stable_genes

# Construct the customized G_svg, G_null, and G_spike sets
sets <- makeCustomGeneSets(
  object = mini_spaDesign2_obj,
  G_svg_base = G_svg_base,
  G_stable = G_stable,
  target_domain = "WM",
  reference_domain = "Layer6"
)

## End(Not run)
```

mini_custom_genes	<i>Mini Custom Gene Sets (WM vs. Layer6)</i>
-------------------	--

Description

The gene-set object returned by [makeCustomGeneSets](#) for the Target/Reference pair "WM" (target) versus "Layer6" (reference), built from the mini DLPFC pilot data. It supplies the labeling, empirical-null, and signal-injection gene sets used by the simulation and testing endpoints.

Usage

```
data(mini_custom_genes)
```

Format

A named list with elements:

G_svg Character vector. Domain-informative + stable genes (labeling set).

G_null Character vector. Stable / empirical-null genes.

G_spike Character vector. Top differentially expressed genes used for signal injection.

TR_stats A data.frame with columns gene, mean_T, mean_R, TR (= mean_T - mean_R), and abs_TR.

Source

See [mini_pilot_data_list](#).

See Also

[makeCustomGeneSets](#), [SaLFC](#), [simulateSpaDesign2](#)

mini_obj_features	<i>Mini spaDesign2 Object (After Feature Selection)</i>
-------------------	---

Description

A [spaDesign2](#) object obtained from [mini_spaDesign2_obj](#) after running [featureSelection](#) and [featureSelectionStable](#). Relative to the unfitted object, its params_expression slot now carries the selected gene sets (top_genes, stable_genes) together with the supporting stable_gene_stats table; the generative parameters (composition, geometry, expression) have not yet been fitted.

Usage

```
data(mini_obj_features)
```

Format

An object of class [spaDesign2](#). The params_expression slot contains top_genes (domain-informative markers), stable_genes (empirical-null / spike candidates), and stable_gene_stats (a data.frame of per-gene stability statistics).

Source

See [mini_pilot_data_list](#).

See Also

[featureSelection](#), [featureSelectionStable](#), [makeCustomGeneSets](#), [mini_obj_fitted](#)

mini_obj_fitted	<i>Mini spaDesign2 Object (Fully Fitted)</i>
-----------------	--

Description

A fully fitted [spaDesign2](#) object obtained from [mini_obj_features](#) after estimating all three generative layers with [estimateCompositionParams_v2](#), [estimateGeometryParams_v2](#), and [estimateExpressionParams_v2](#). It is ready to drive synthetic-data generation ([simulateSpaDesign2](#)) and power evaluation.

Usage

```
data(mini_obj_fitted)
```

Format

An object of class [spaDesign2](#) with all parameter slots populated:

`params_composition` Baseline-anchored Binomial-logit fit, including `beta_binomial` with intercept `beta0` and effect `beta1`.

`params_geometry` Fitted Fisher-Gaussian kernel mixture (FGKMM) geometry parameters.

`params_expression` Gene sets plus the fitted expression parameters, e.g. domain means (`mu_grand`), spatial covariance (`theta`), and between-sample variance (`sigma_bio`).

Source

See [mini_pilot_data_list](#).

See Also

[estimateExpressionParams_v2](#), [simulateSpaDesign2](#), [pBANSKY](#)

mini_pilot_data_list *Mini DLPFC Pilot Data List*

Description

A small, downsampled list of human dorsolateral prefrontal cortex (DLPFC) spatial transcriptomics pilot samples, in the input format expected by [createSpaDesign2Object](#).

Usage

```
data(mini_pilot_data_list)
```

Format

A named list of pilot samples (DLPFC sections, e.g. "151507"–"151509"). Each element is itself a list with:

counts A genes $\times$ spots count matrix (reduced gene panel).

coords A data.frame of spot-level metadata with columns x, y (spatial coordinates) and domain (annotated spatial domain, e.g. "Layer1"–"Layer6" and "WM").

group Experimental group label (control = 0, case = 1).

sample_id Character sample identifier.

Source

Derived from the human DLPFC 10x Visium data of Maynard et al. (2021), *Nature Neuroscience* 24:425–436, obtained via the **spatialLIBD** package (Pardo et al., 2022), *BMC Genomics* 23:434.

See Also

[createSpaDesign2Object](#)

mini_spasDesign2_obj *Mini spaDesign2 Object (Created, Unfitted)*

Description

A [spaDesign2](#) object built from [mini_pilot_data_list](#) via [createSpaDesign2Object](#). The pilot_data slot is populated, but no parameters have been estimated yet, so params_composition, params_geometry, params_expression, synthetic_data, and testing_result are still empty. It is the natural starting point for feature selection and parameter estimation.

Usage

```
data(mini_spasDesign2_obj)
```

Format

An object of class [spaDesign2](#) with the pilot_data slot populated and the remaining slots empty.

Source

See [mini_pilot_data_list](#).

See Also

[createSpaDesign20bject](#), [featureSelection](#), [mini_obj_features](#), [mini_obj_fitted](#)

pBANSKY

Pilot-Guided Spatial Clustering (pBANSKY, Optimized)

Description

Pilot-centroid-guided spatial clustering on a robust three-block BANKSY embedding (own expression C , neighbourhood mean M , and neighbourhood gradient G). Latent truth may be supplied in `coords$tilde_d` or `coords$domain`; recovered labels are written to `coords$hat_d`.

The recovered labels are statistically equivalent to a reference loop-based implementation (up to floating-point reassociation and PCA sign conventions, neither of which affects the labels), but substantially faster. The main algorithmic gains include:

1. The per-spot neighbour-mean (M) and neighbour-gradient (G) loops become sparse products with the row-normalized neighbour-weight matrix W , via $M = CW$ and $G^2 = (C \odot C)W - 2C \odot (CW) + C \odot C$ (then an elementwise square root and re-scaling).
2. PCA uses a truncated SVD (`RSpectra::svds`) for the top `n_pcs` components, falling back to `stats::prcomp` when **RSpectra** is unavailable.
3. The zero-variance row filter is fully vectorized.
4. Samples are processed in parallel across `n_cores`; the shared pilot embedding is read-only.

Usage

```
pBANSKY(
  sim_data,
  pilot_data,
  lambda,
  k_neighbors = 15,
  verbose = TRUE,
  use_G = TRUE,
  balance_mode = c("energy_proportional", "equal_split", "paper_mu"),
  mu = 1.5,
  scale_pilot_coords = TRUE,
  scale_sim_coords = TRUE,
  hard_guidance = FALSE,
  n_pcs = 20,
  do_hungarian = FALSE,
  n_cores = 1L
)
```

Arguments

sim_data	List of simulated samples (each with counts or logcounts and a coords data frame); typically the output of simulateSpaDesign2 .
pilot_data	List of pilot samples. The sample with the most domains (then the most spots) seeds the cluster centroids.
lambda	Numeric in $[0, 1]$. BANKSY spatial-mixing weight (0 = expression only, 1 = neighbourhood only).
k_neighbors	Integer. Number of spatial neighbours in the BANKSY graph. Default 15.
verbose	Logical. Print progress. Default TRUE.
use_G	Logical. Include the neighbourhood-gradient (G) block. Default TRUE.
balance_mode	Character. How the spatial budget is split between the M and G blocks: one of "energy_proportional" (default), "equal_split", or "paper_mu".
mu	Numeric. Gradient down-weighting, used only when balance_mode = "paper_mu". Default 1.5.
scale_pilot_coords	Logical. Min-max scale the pilot coordinates to the unit square before graph construction. Default TRUE.
scale_sim_coords	Logical. Min-max scale the simulated coordinates. Default TRUE.
hard_guidance	Logical. If TRUE, assign each spot to its nearest pilot centroid instead of running k-means. Default FALSE.
n_pcs	Integer. Number of principal components of the joint embedding. Default 20.
do_hungarian	Logical. If TRUE and latent truth is available, align recovered labels to truth via the Hungarian algorithm. Default FALSE.
n_cores	Integer. Samples processed in parallel (clamped to 1 on Windows). Default 1.

Value

A list of processed samples. Each element carries the input counts, a coords data frame with the recovered domain in `hat_d` (and a per-spot `is_correct` flag when truth is available), plus clustering diagnostics (`global_accuracy`, `n_domains`).

plot,spaDesign2,missing-method

Plot Spatial Domains for a spaDesign2 Object

Description

A unified plotting method for spaDesign2 objects. Visualizes the spatial distribution of domains for both pilot data and synthetic data. It automatically extracts unified domain levels so that colours are consistent across all plots.

Usage

```
## S4 method for signature 'spaDesign2,missing'
plot(
  x,
  y,
  type = c("pilot", "synthetic_raw", "synthetic_mapped"),
  color_by = c("hat_d", "domain"),
  n_per_group = 3,
  ncol = 3,
  sample_ids = NULL,
  cex = 1,
  alpha = 0.9,
  title = NULL,
  subtitle = NULL,
  colors = NULL,
  reverse_y = FALSE,
  ...
)
```

Arguments

x	A spaDesign2 object.
y	Not used. Required for S4 generic compatibility.
type	Character. Which data to plot: "pilot" (ground truth), "synthetic_raw" (before rearrangement, continuous), or "synthetic_mapped" (after rearrangement, KD-tree snapped).
color_by	Character. For synthetic data, colour by the recovered domains ("hat_d", default) or by truth ("domain").
n_per_group	Integer. Number of samples to show per group. Default 3.
ncol	Integer. Number of columns in the facet wrap. Default 3.
sample_ids	Optional character vector of specific sample IDs to plot.
cex	Numeric. Point-size expansion factor. Default 1.0.
alpha	Numeric. Point transparency in [0, 1]. Default 0.9.
title	Character. Custom plot title; if NULL, a default is used.
subtitle	Character. Custom subtitle; if NULL, a default is used.
colors	Character vector of colours. If NULL, uses the "Set1" palette.
reverse_y	Logical. If TRUE, reverses the y-axis to match image coordinates. Default FALSE.
...	Additional arguments (currently ignored).

Value

A ggplot object.

Examples

```
## Not run:
data("mini_obj_mapped", package = "spaDesign2")

# Using a custom color palette
```

```
my_colors <- c("NEURAL" = "#E8792B", "CARDIOMUSCULAR" = "#D42A2F",
              "LIVER" = "#2CA02C", "LUNG" = "#2D6BB4", "OTHER" = "#D3D3D3")

# Plot with Y-axis reversed (for image-based coordinates)
plot(mini_obj_mapped, type = "pilot", colors = my_colors, reverse_y = TRUE)

## End(Not run)
```

plot.spaDesign2Tuning *Plot Method for spaDesign2Tuning*

Description

Plot Method for spaDesign2Tuning

Usage

```
## S3 method for class 'spaDesign2Tuning'
plot(x, ...)
```

Arguments

x	An object of class spaDesign2Tuning.
...	Additional arguments to be passed to plotting functions.

plotPowerCurveLOR *Plot Composition (LOR) Power / Size Curve*

Description

Smooths the empirical rejection rate (= power for effect > 0, = size at effect = 0) against **K**, one curve per composition effect size, with a dashed reference line at the nominal level alpha. Uses a shape-constrained additive model (monotone-increasing in **K**, `scam`, `bs = "mpi"`) when **scam** is installed and there are ≥ 4 distinct **K**; otherwise falls back to monotone isotonic interpolation.

Usage

```
plotPowerCurveLOR(summary_df, alpha = 0.05, k_spline = 5)
```

Arguments

summary_df	Data frame from evaluatePowerLOR.
alpha	Numeric. Nominal level (reference line + title).
k_spline	Integer. Spline basis dimension (capped at #unique K).

Value

A ggplot object.

See Also

[evaluatePowerLOR](#)

plotPowerCurveSaLFC	<i>Plot SCAM-Smoothed Power Curve</i>
---------------------	---------------------------------------

Description

Smooths the empirical power surface with a shape-constrained additive model (monotone-increasing in K per effect size, bs = "mpi"). Falls back to mgcv (bs = "ps", no monotonicity) with a warning if **scam** is unavailable.

Usage

```
plotPowerCurveSaLFC(summary_df, alpha = 0.05, k_spline = 5)
```

Arguments

summary_df	Data frame from evaluatePowerSaLFC.
alpha	Numeric. Significance level (for the title).
k_spline	Integer. Spline basis dimension (capped at #unique K).

Value

A ggplot object.

See Also

[evaluatePowerSaLFC](#)

print.powerLOR	<i>Print a LOR composition power grid (compact)</i>
----------------	---

Description

Print a LOR composition power grid (compact)

Usage

```
## S3 method for class 'powerLOR'
print(x, digits = 3, ...)
```

Arguments

x	A powerLOR object.
digits	Rounding for the rate matrix.
...	Ignored.

Value

x, invisibly.

print.powerSaLFC	<i>Print a SaLFC power grid (compact)</i>
------------------	---

Description

Print a SaLFC power grid (compact)

Usage

```
## S3 method for class 'powerSaLFC'
print(x, digits = 3, ...)
```

Arguments

x	A powerSaLFC object.
digits	Rounding for the power matrix.
...	Ignored.

Value

x, invisibly.

rearrangeSyntheticToPilot	<i>Rearrange Synthetic Spots onto Pilot Domain Geometry (Per-Domain OT)</i>
---------------------------	---

Description

Reshapes blob-like simulated domains so that each domain occupies the same spatial region as the corresponding domain in a reference pilot sample, via a two-stage per-domain optimal transport (exact 2D-Gaussian affine alignment, then bijective nearest-neighbour snapping onto real pilot spots).

Composition-aware mode (`boundary_shift = TRUE`). The default per-domain map sends each synthetic domain onto the SAME domain's pilot footprint, so an increased case-group target proportion is rendered as higher *density* inside the fixed (control) pilot target footprint rather than a larger *area*. When `boundary_shift = TRUE` (with `target_domain` and `reference_domain` supplied), the target and reference pilot spots are pooled and re-partitioned per section so that the target **territory area fraction matches that section's own target proportion** $f = n_{target} / (n_{target} + n_{reference})$: the target region is grown contiguously into the nearest reference spots until it holds an f fraction of the pool. Synthetic target spots are then placed in this (enlarged, for case) territory and reference spots in the remainder. Labels and per-domain counts are untouched (so the LOR statistic is unchanged); only the spatial footprint reflects the effect. Control sections (baseline f) look like the pilot; case sections show the target band expanding into the reference. All other domains map by the standard per-domain OT.

Usage

```
rearrangeSyntheticToPilot(
  synthetic_data,
  pilot_data,
  match_by = c("hat_d", "domain"),
  jitter_frac = 0.4,
  k_nn = 80L,
  seed = 1L,
  verbose = TRUE,
  boundary_shift = FALSE,
  target_domain = NULL,
  reference_domain = NULL
)
```

Arguments

<code>synthetic_data</code>	List of simulated samples; each has coords with x, y, the <code>match_by</code> label, and <code>meta\$group/meta\$sample_id</code> .
<code>pilot_data</code>	List of pilot samples; each has coords with x, y, domain, group, <code>sample_id</code> .
<code>match_by</code>	"hat_d" (default) or "domain".
<code>jitter_frac</code>	Numeric. Oversampling jitter as a fraction of local spacing. Default 0.4.
<code>k_nn</code>	Integer. Candidate neighbours for the greedy assignment. Default 80.
<code>seed</code>	Integer base seed (global RNG restored on exit). Default 1.
<code>verbose</code>	Logical. Default TRUE.
<code>boundary_shift</code>	Logical. If TRUE, render the target/reference composition difference as a moving boundary (see Description). Default FALSE (original per-domain behaviour, unchanged).
<code>target_domain, reference_domain</code>	Character. The domain pair whose boundary moves when <code>boundary_shift = TRUE</code> (e.g. "WM" and "Layer6"). Must match the labels in the <code>match_by</code> column.

Value

The input list with `coords$x/coords$y` replaced by the rearranged coordinates; originals kept as `x_raw/y_raw`, and `matched_pilot_id` records the reference section.

See Also

[pBANSKY](#), [simulateSpaDesign2](#)

Description

Tests for a differential Target-vs-Reference log-fold-change between two groups of spatial samples, using ground-truth domain labels. Per sample, the statistic $z = \bar{x}_T - \bar{x}_R$ is formed together with its spatial sampling variance τ^2 (derived from an exponential covariance with the per-gene parameters in `object@params_expression`). Across samples, the biological variance is estimated, optionally moderated via Smyth-style empirical Bayes (`limma::squeezeVar`), and a TREAT-style threshold test against `lfc_threshold` is performed with Satterthwaite degrees of freedom. A rank-based (Wilcoxon) alternative is also available.

Usage

```
SaLFC(
  processed_data,
  object,
  genes,
  target_domain = "WM",
  reference_domain = NULL,
  domain_col = c("hat_d", "domain"),
  rho_TR = 0,
  adjust_method = c("bonferroni", "BH"),
  test_method = c("wilcoxon", "t"),
  alternative = c("greater", "less", "two.sided"),
  lfc_threshold = 0,
  pilot_threshold_args = list(),
  max_spots_per_domain = 5000L,
  seed = 2026L
)
```

Arguments

<code>processed_data</code>	List of samples (e.g. recovered synthetic data). Each element must contain counts (genes x spots) and coords with columns x, y, and the label column selected by <code>domain_col</code> (<code>hat_d</code> and/or <code>domain</code>); optionally <code>meta\$group</code> and <code>meta\$sample_id</code> .
<code>object</code>	A <code>spaDesign2</code> object, used for per-gene spatial parameters (<code>theta</code>) via <code>object@params_expression</code> .
<code>genes</code>	Character vector of genes to test.
<code>target_domain</code>	Character. Target domain label.
<code>reference_domain</code>	Character or NULL. Reference domain; if NULL, all non-target spots are used as the reference.
<code>domain_col</code>	Character. Which coords column defines the target and reference domains: <code>"hat_d"</code> (recovered clusters, default) or <code>"domain"</code> (ground truth). Falls back to <code>"domain"</code> when the requested column is absent (e.g. pilot data, which carries only ground-truth labels).
<code>rho_TR</code>	Numeric. Assumed Target-Reference biological correlation (< 1).
<code>adjust_method</code>	Multiple-testing correction: <code>"BH"</code> (FDR, default) or <code>"bonferroni"</code> (FWER).
<code>test_method</code>	Group comparison on per-sample <code>z_hat</code> : <code>"wilcoxon"</code> or <code>"t"</code> .
<code>alternative</code>	Test direction: <code>"greater"</code> (default), <code>"less"</code> , or <code>"two.sided"</code> .

lfc_threshold Numeric scalar, named numeric vector keyed by gene, or the string "pilot". If a named vector, an optional "global_tau" attribute supplies the fallback for missing/non-finite entries. If "pilot", the threshold is estimated on the fly from `object@pilot_data` via `estimate_lfc_threshold_from_pilot()` and then used for the test.

pilot_threshold_args Named list of extra arguments forwarded to `estimate_lfc_threshold_from_pilot()` when `lfc_threshold = "pilot"` (e.g. `list(q = 0.75, min_tau = 0.05)`). `object`, `genes`, `target_domain`, `reference_domain`, `rho_TR`, `adjust_method`, `test_method`, and `alternative` are supplied automatically but may be overridden here.

max_spots_per_domain Integer. Per-domain spot cap; domains larger than this are randomly down-sampled (bounds the $O(n^2)$ distance matrices). The global RNG state is restored on exit.

seed Integer base seed for reproducible down-sampling.

Value

A data.frame of per-gene moderated statistics and adjusted p-values, ordered by `padj` then `p_value`. An empty data.frame is returned when no gene has sufficient data.

See Also

[evaluatePowerSaLFC](#), [estimate_lfc_threshold_from_pilot](#)

Examples

```
## Not run:
data("mini_obj_fitted", package = "spaDesign2")
data("mini_custom_genes", package = "spaDesign2")

## One-point SaLFC power calculation (single design point: K = 5, DE_lfc = 0.3):
## simulate -> recover domains -> rearrange -> SaLFC -> rejection rate.
sim <- simulateSpaDesign2(mini_obj_fitted, n_sample_per_group = 5,
  scenario_settings = list(DE_lfc = 0.3), target_domain = "WM",
  genes_to_simulate = mini_custom_genes$G_svg,
  de_genes = mini_custom_genes$G_spike, seed_base = 1)

syn <- pBANSKY(sim, mini_obj_fitted@pilot_data, lambda = 0.5, do_hungarian = TRUE)
syn <- setNames(lapply(syn, function(s) list(counts = s$counts, coords = s$coords,
  meta = data.frame(sample_id = s$sample_id, group = s$group))),
  vapply(syn, function(s) s$sample_id, character(1)))
syn <- rearrangeSyntheticToPilot(syn, mini_obj_fitted@pilot_data, match_by = "hat_d")

res <- SaLFC(syn, mini_obj_fitted,
  genes = unique(c(mini_custom_genes$G_null, mini_custom_genes$G_spike)),
  target_domain = "WM", reference_domain = "Layer6", domain_col = "hat_d",
  lfc_threshold = "pilot")

## empirical power at this point = fraction of true spike-ins at FDR <= 0.05
mean(res$padj[res$gene %in% mini_custom_genes$G_spike] <= 0.05)

## End(Not run)
```

simulateSpaDesign2	<i>Simulate Spatial Transcriptomics Data (Hybrid Mode, Optimized)</i>
--------------------	---

Description

Optimized rewrite of simulateSpaDesign2. Statistically identical to the previous version under the same seed_base, but substantially faster. The main algorithmic gains include:

- The conditional-texture (eta_cond) KNN structure is built once per domain per sample and reused across all genes.
- Sample/group invariants are hoisted out of the per-sample and per-gene loops.
- The per-spot domain-mean lookup is fully vectorized.

Usage

```
simulateSpaDesign2(
  object,
  n_sample_per_group = 10,
  scenario_settings = list(DE_lfc = 0, target_prop_case = NULL, delta_pp = NULL, kappa_m
    = 1, kappa_bio = 1, delta_rho = 1, lambda_cond = 0.1),
  target_domain = NULL,
  genes_to_simulate = NULL,
  de_genes = NULL,
  verbose = TRUE,
  n_cores = 4,
  gp_rank = 20L,
  gp_jitter = 1e-08,
  spatial_mode = c("original", "basis"),
  graph_k = 20L,
  cond_k_nn = 20L,
  basis_rank = 25L,
  basis_kernel = c("gaussian", "exponential"),
  basis_knot_method = c("sample", "grid"),
  basis_seed = 1L,
  lambda_cloud = NULL,
  seed_base = NULL
)
```

Arguments

object	A spaDesign2 object.
n_sample_per_group	Integer. Number of synthetic samples to generate per group.
scenario_settings	List. Scenario configurations including DE_lfc, target_prop_case, kappa_m, kappa_bio, delta_rho, and lambda_cond.
target_domain	Character. Target domain name for differential effects.
genes_to_simulate	Character vector. Genes to simulate.

de_genes	Character vector. Subset of genes that will have DE effects applied.
verbose	Logical. Print progress.
n_cores	Integer. Number of cores for parallel execution.
gp_rank	Integer. Rank for Gaussian Process (legacy).
gp_jitter	Numeric. Jitter to ensure positive definiteness.
spatial_mode	Character. "original" or "basis".
graph_k	Integer. KNN for the whole-tissue spatial graph.
cond_k_nn	Integer. KNN for the pilot-guided conditional texture.
basis_rank	Integer. Basis rank for spatial mode "basis".
basis_kernel	Character. "gaussian" or "exponential".
basis_knot_method	Character. "sample" or "grid".
basis_seed	Integer. Seed for basis knot sampling.
lambda_cloud	Optional numeric in $[0, 1]$. Used for FGKMM simulation.
seed_base	Optional integer for reproducible simulations.

Value

A list of simulated synthetic samples.

Examples

```
## Not run:
# Load the pre-computed mini object and custom gene sets
data("mini_obj_fitted", package = "spaDesign2")
data("mini_custom_genes", package = "spaDesign2")

scenario_params <- list(
  DE_lfc = 0.0,
  target_prop_case = 0.7,
  kappa_geo = 1.0,
  kappa_m = 1.0,
  kappa_bio = 1.1,
  delta_rho = 1.0,
  lambda_cond = 0.5
)

sim_results <- simulateSpaDesign2(
  object = mini_obj_fitted,
  n_sample_per_group = 5,
  scenario_settings = scenario_params,
  target_domain = "WM",
  genes_to_simulate = mini_custom_genes$G_svg,
  de_genes = mini_custom_genes$G_spike,
  spatial_mode = "original",
  graph_k = 30L,
  verbose = TRUE,
  n_cores = 1
)

## End(Not run)
```

spaDesign2-class	<i>spaDesign2 Class</i>
------------------	-------------------------

Description

S4 class for multi-sample spatial transcriptomics power analysis. Stores the pilot data, the hierarchical model parameters fitted from it, the simulated synthetic data, and the downstream testing results.

Slots

`pilot_data` list. Pilot samples; each element contains counts (genes x spots), coords (x, y, domain), group (0/1), and `sample_id`.

`params_geometry` list. Hierarchical FGKMM parameters (centroids, shapes, pooling info).

`params_expression` list. Hierarchical NNGP parameters (mean, covariance, `sigma_bio`).

`params_composition` list. Beta-Binomial parameters for domain proportions.

`synthetic_data` list. Simulated/recovered samples produced by the simulation and clustering pipeline. Each element holds counts, coords, and meta.

`testing_result` list. Endpoint testing results, keyed by endpoint (e.g., `salfc`, `composition`).

<code>summary.powerLOR</code>	<i>Summarize a LOR composition power analysis (lm-style report)</i>
-------------------------------	---

Description

Produces an interpretive summary: a rejection-rate table over K x effect, the minimum K reaching `target_power` per (non-null) effect (grid + continuous estimate), and the empirical size at the null.

Usage

```
## S3 method for class 'powerLOR'
summary(object, target_power = 0.8, ...)
```

Arguments

<code>object</code>	A <code>powerLOR</code> object.
<code>target_power</code>	Numeric in (0,1).
<code>...</code>	Ignored.

Value

An object of class `summary.powerLOR`.

summary.powerSaLFC	<i>Summarize a SaLFC power analysis (lm-style report)</i>
--------------------	---

Description

Produces an interpretive summary: a power table over $K \times DE_lfc$, the minimum K reaching `target_power` per effect (grid value plus a monotone-interpolated continuous estimate), and an FDR-control diagnostic.

Usage

```
## S3 method for class 'powerSaLFC'
summary(object, target_power = 0.8, ...)
```

Arguments

<code>object</code>	A powerSaLFC object.
<code>target_power</code>	Numeric in (0,1).
<code>...</code>	Ignored.

Value

An object of class `summary.powerSaLFC` (a list with `$by_cell`, `$power`, `$se`, `$min_k`, `$fdr`, `$design`); printed as a report.

summary.spaDesign2Tuning	<i>Summary Method for spaDesign2Tuning</i>
--------------------------	--

Description

Summary Method for spaDesign2Tuning

Usage

```
## S3 method for class 'spaDesign2Tuning'
summary(object, ...)
```

Arguments

<code>object</code>	An object of class <code>spaDesign2Tuning</code> .
<code>...</code>	Additional arguments affecting the summary produced.

syntheticData	<i>Get or Set Synthetic Data</i>
---------------	----------------------------------

Description

Accessor and replacement methods for the `synthetic_data` slot of a `spaDesign2` object.

Usage

```
syntheticData(object)

## S4 method for signature 'spaDesign2'
syntheticData(object)

syntheticData(object) <- value

## S4 replacement method for signature 'spaDesign2'
syntheticData(object) <- value
```

Arguments

<code>object</code>	A <code>spaDesign2</code> object.
<code>value</code>	A named list of simulated samples to store.

Value

`syntheticData` returns the stored list; the replacement method returns the updated `spaDesign2` object.

testingResult	<i>Get or Set Testing Results</i>
---------------	-----------------------------------

Description

Accessor and replacement methods for the `testing_result` slot of a `spaDesign2` object.

Usage

```
testingResult(object)

## S4 method for signature 'spaDesign2'
testingResult(object)

testingResult(object) <- value

## S4 replacement method for signature 'spaDesign2'
testingResult(object) <- value
```

Arguments

object	A spaDesign2 object.
value	A named list of endpoint results to store.

Value

testingResult returns the stored list; the replacement method returns the updated spaDesign2 object.

tuneSpaDesignLambdas *Stability- and Validity-Driven Hyperparameter Tuning*

Description

Tunes the spatial hyperparameters of a spaDesign2 object so that the resulting simulation behaves as a *faithful and stable power oracle*. The recovery weight `lambda_p` (pBANKSY) is tuned once by maximizing clustering reproducibility (ARI), exactly as before. The conditional-texture weight `lambda_cond` is tuned with a regularized-noncentrality objective and a shared cross-effect selection rule that fix two defects of a pure $Var(T)$ -minimization:

1. **Objective.** Power is governed by the noncentrality $\delta = E[T]/sd[T]$, not by $sd[T]$ alone. Minimizing variance can be achieved by attenuating the signal, which collapses power. The objective is therefore a regularized noncentrality (equivalently the reciprocal of a floored coefficient of variation),

$$\hat{\delta}(\lambda) = \bar{T}/(sd(T) + \varepsilon s_0),$$

which is scale-free and cannot be gamed by uniform shrinkage. Both endpoints are placed on the *same* standardized (t-statistic) footing: the SaLFC endpoint uses `mean(abs(t_stat))` and the LOR endpoint uses the standardized statistic `LOR()$stat` (not the raw `delta_hat`).

2. **Effect ordering.** Tuning `lambda_cond` independently per effect size leaves nothing to enforce that the realized noncentrality increases with effect size, so higher-effect power curves can fall below lower-effect ones. A single `lambda_cond` is therefore selected jointly across the effect grid (shared), maximizing the worst-case noncentrality across effects; under a shared weight the realized noncentrality inherits the generator's monotonicity in effect size. The realized curve is isotonized (PAVA) for reporting.

A null-calibration guard (`calibration_guard`) further down-weights any `lambda_cond` whose empirical type-I error under the matched null (SaLFC: `DE_lfc = 0`, no DE genes; LOR: `target_prop_case = p0`, the pilot baseline proportion) is inflated, so noncentrality is never bought by biasing the generator.

Usage

```
tuneSpaDesignLambdas(
  object,
  target_domain,
  reference_domain,
  G_DE,
  scenario_H1,
  lambda_p_grid = seq(0.2, 0.8, 0.1),
  lambda_c_grid = seq(0.2, 0.8, 0.1),
```

```

B = 10,
K_syn = 10,
n_cores = 1,
seed = 2026,
verbose = TRUE,
eps_floor = 0.001,
calibration_guard = TRUE,
alpha = 0.05,
calib_mult = 2
)

```

Arguments

object	A spaDesign2 object.
target_domain	Character. Target domain name.
reference_domain	Character. Reference domain name.
G_DE	Character vector. Genes with a true DE effect (H1 scenario).
scenario_H1	List. Joint scenario settings for H1. DE_lfc and target_prop_case may each be a scalar or a numeric vector; when a vector, lambda_cond is evaluated at every value and a single shared value is selected across the grid.
lambda_p_grid	Numeric vector. Grid of lambda_p to test.
lambda_c_grid	Numeric vector. Grid of lambda_cond to test.
B	Integer. Number of replicates per grid point (Default: 10).
K_syn	Integer. Number of samples per group during tuning (Default: 10).
n_cores	Integer. Number of cores for simulation.
seed	Integer. Random seed for reproducible tuning. Default is 2026.
verbose	Logical. Prints progress.
eps_floor	Numeric. Denominator floor multiplier ε for the regularized noncentrality (Default: 1e-3). Stabilizes the $E[T] \rightarrow 0$ regime (relevant for LOR near the null).
calibration_guard	Logical. If TRUE (default) compute the null-calibration weight and down-weight inflated lambda_cond.
alpha	Numeric. Nominal significance level for the calibration guard.
calib_mult	Numeric > 1. Tolerance multiplier: the calibration weight is ≈ 1 while the empirical type-I error stays below $\alpha * \text{calib_mult}$ and decays smoothly beyond it.

Value

An object of class `spaDesign2Tuning`. `optimal_params` has one entry per endpoint (SaLFC, LOR); each contains the shared `lambda_p`, a representative scalar `lambda_cond` (at the median effect), a `by_effect` data frame (tuned `lambda_cond`, realized noncentrality SNR, its isotonic projection `SNR_iso`, and the calibration weight), and the shared `lambda_cond`. `summary_stage2` holds the per-endpoint noncentrality surfaces (with `mean_T`, `sd_T`, SNR, and `Var_T` for reference). `diagnostics` stores the calibration table.

Examples

```
## Not run:
data("mini_obj_fitted", package = "spaDesign2")
tuning_H1_scenario <- list(
  DE_lfc = c(0.30, 0.35, 0.40), target_prop_case = c(0.65, 0.70, 0.75),
  kappa_geo = 1.0, kappa_m = 1.0, kappa_bio = 1.0, delta_rho = 1.0
)
tuning_obj <- tuneSpaDesignLambdas(
  object = mini_obj_fitted, target_domain = "WM", reference_domain = "Layer6",
  G_DE = sets$G_spike, scenario_H1 = tuning_H1_scenario,
  lambda_p_grid = seq(0.2, 0.8, by = 0.1),
  lambda_c_grid = seq(0.2, 0.8, by = 0.1),
  B = 10, K_syn = 5, n_cores = 1, seed = 123
)
summary(tuning_obj); plot(tuning_obj)
tuning_obj$optimal_params$SaLFC$by_effect
tuning_obj$optimal_params$LOR$by_effect

## End(Not run)
```

Index

* datasets

- mini_custom_genes, [21](#)
- mini_obj_features, [21](#)
- mini_obj_fitted, [22](#)
- mini_pilot_data_list, [23](#)
- mini_spaDesign2_obj, [23](#)

addTestingResult, [3](#)

as_powerLOR, [3](#)

as_powerSaLFC, [4](#)

createSpaDesign2Object, [4](#), [23](#), [24](#)

estimate_lfc_threshold_from_pilot, [8](#),
[32](#)

estimate_lor_tau_from_pilot, [10](#), [19](#)

estimateCompositionParams_v2, [5](#), [22](#)

estimateExpressionParams_v2, [6](#), [22](#)

estimateGeometryParams_v2, [7](#), [22](#)

evaluatePowerLOR, [11](#), [15](#), [19](#), [27](#)

evaluatePowerSaLFC, [13](#), [13](#), [28](#), [32](#)

featureSelection, [15](#), [21](#), [22](#), [24](#)

featureSelectionStable, [16](#), [21](#), [22](#)

LOR, [10](#), [11](#), [13](#), [17](#)

makeCustomGeneSets, [19](#), [21](#), [22](#)

mini_custom_genes, [21](#)

mini_obj_features, [21](#), [22](#), [24](#)

mini_obj_fitted, [22](#), [22](#), [24](#)

mini_pilot_data_list, [21–23](#), [23](#), [24](#)

mini_spaDesign2_obj, [21](#), [23](#)

pBANSKY, [22](#), [24](#), [30](#)

plot, spaDesign2, missing-method, [25](#)

plot.spaDesign2Tuning, [27](#)

plotPowerCurveLOR, [13](#), [27](#)

plotPowerCurveSaLFC, [15](#), [28](#)

print.powerLOR, [13](#), [28](#)

print.powerSaLFC, [15](#), [29](#)

rearrangeSyntheticToPilot, [29](#)

SaLFC, [9](#), [15](#), [21](#), [30](#)

simulateSpaDesign2, [21](#), [22](#), [25](#), [30](#), [33](#)

spaDesign2, [21–23](#)

spaDesign2-class, [35](#)

summary.powerLOR, [13](#), [35](#)

summary.powerSaLFC, [15](#), [36](#)

summary.spaDesign2Tuning, [36](#)

syntheticData, [37](#)

syntheticData, spaDesign2-method
(syntheticData), [37](#)

syntheticData<- (syntheticData), [37](#)

syntheticData<- , spaDesign2-method
(syntheticData), [37](#)

testingResult, [37](#)

testingResult, spaDesign2-method
(testingResult), [37](#)

testingResult<- (testingResult), [37](#)

testingResult<- , spaDesign2-method
(testingResult), [37](#)

tuneSpaDesignLambdas, [13](#), [15](#), [38](#)